GentleVanguard

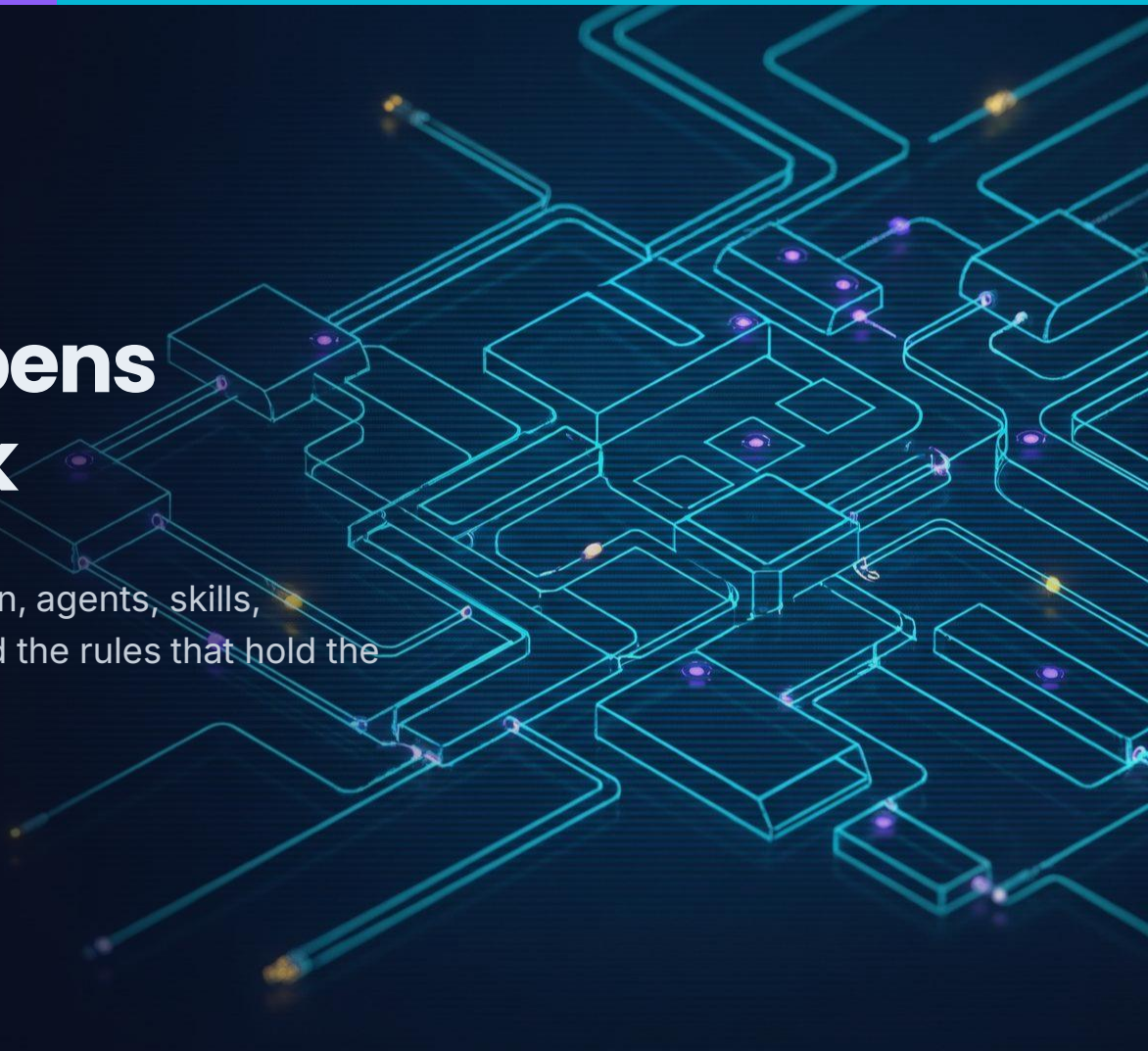
TECHNICAL ONBOARDING

How work happens inside the stack

Repository map, anatomy of a session, agents, skills, commands, operational database and the rules that hold the rest together.

Stack v4.3.0 | Node 24 | pnpm 11

Onboarding guide - November 2026



ROUTE

What you will walk through

The recommended order for a first week.

01 **Repository map**
What lives in each folder and why it is separated.

02 **Anatomy of a session**
What happens between your prompt and the agent reply.

03 **Agents and skills**
The 21 roles in the roster and how capabilities get invoked.

04 **Commands**
The commands used daily and what each one verifies.

05 **Nexus**
The operational database: 43 tables, migrations and backup.

06 **Development flow**
From request to commit: ODD, slices, gates and the reviewed boundary.

07 **Repo rules**
What is not negotiable and why it exists.

08 **Where to look**
The shortest path to each kind of answer.

MAP

Eight folders, one responsibility each

The separation is not cosmetic: it is what makes the validations automatic.

src/	786 TypeScript files. Stack code by area: orchestration, security, retrieval, data, tools, quality and ops.
apps/	28 of our own applications running on the stack, validating the architecture in production.
skills/	250 capabilities with contract and frontmatter. Invoked by task description.
docs/	Stack manual by module, full manual, 38 ADRs and the design decision log.
config/	Single source of configuration: model router, role mapping, quality gates, budgets and MCP registry.
packages/	Workspace holding the official design system and the shared packages the code imports.
tests/	272 test files: unit, e2e, integration, security, skills, workflows, eval and performance.
assets/	Official brand, self-hosted fonts and the canonical shell diagram.

What happens when you write a request

Six stages, all instrumented. None of them is magic: each one leaves a record.

Autostart

Prepares session, engram, security orchestrator, codegraph, budgets, watchtower and Nexus.

>

Pre-processing

Injects skills, context and rules before the model sees the message.

>

Router

Picks the model by cost, latency, domain and task criticality.

>

Agent

Executes with real tools. Every step lands in the trace.

>

Skills

Invoked by context. Each skill declares its contract and its outcome.

>

Nexus

Sessions, traces, tokens, events and skill outcomes are persisted.

ROSTER

21 agents, each with its own perimeter

An agent has a model, permissions, a step limit, a budget and a declared fallback.



Orchestration

orchestrator, session-agent, knowledge-agent and self-diag-agent: the loop and its own diagnostics.



SDD cycle

sdd-explore, sdd-design, sdd-apply and sdd-verify: the opt-in path for multi-session specs.



Operations

ops-agent, maintenance-agent, gitflow-agent, premortem-agent and doc-agent: daily operation.



Business areas

mkt, sales, finance, hr, legal, bus-tele and sia: business roles with their own permissions.

Skills: the library that stops us rewriting things

What they are

A SKILL.md with frontmatter, description and contract. They are indexed and routed by task similarity, not by remembering the name.

How they are invoked

The skill router scores the task against every capability description and proposes the closest one.

How they are validated

24 critical contracts are exercised in real execution. Nexus separates production, evaluation and maintenance outcomes.

What is not copied

Only 28 critical skills propagate to the five external surfaces. The rest are not copied so host metadata does not degrade.

What was adopted

Including frontend-design, canvas-design, doc-coauthoring, theme-factory and the marketing family, with attribution in the frontmatter.

Hygiene

Every skill passes structure and accent lint plus the secret scanner before it enters the index.

The ones used every day

Every stack operation is an auditable command. There are no hidden actions.

Command	What it does
npm run session:autostart:detached	Session startup in the background. Idempotent. About 1.3 seconds.
npm run watchtower:health	154 checks over 29 components. PASS expected.
npm run db:health	Nexus verification: integrity, WAL, tables and rows.
npm run token:status	Real budget: used, ceiling and percentage.
npm run stack:facts	Regenerates the stack fact inventory and its documentation.
npm run stack:facts:check	Fails if there is structural drift against the inventory.
npm run quality:universal	Universal quality gate: intake, plan, build, verify, release, operate, learn.
npm run guard:snapshot	Backs up untracked files and uncommitted patches before a risky operation.
npm run scan:secrets -- --scan	80 detection patterns, with optional entropy and redaction.
npm run graphify -- query "..."	Query the deterministic AST graph of the repository.
npx tsx src/orchestration/recommend-agent.ts --task "..." --topn 3	Agent recommendation for a task.
npm run upgrade	Syncs the workspace with what is published on origin, with a prior snapshot.

Nexus: the operational memory of the stack

43 tables and 22 migrations in SQLite with WAL. Everything the stack learns ends up here.

What it holds

Metrics, sessions, traces, events, alerts, feedback, cache, skill usage, tokens and routing.

Retention

Events at 30 days, cache at 7 and token usage at 90. Daily backups keeping 14 copies.

Queries

Direct SQL over the database, with no reporting layer that could drift from the truth.

Integrity

The watchtower checks file size, WAL, integrity and row counts on every health cycle.

Engram	Persistent memory across sessions
CodeGraph	Symbols and relations in the repository
Nexus	Metrics, events and traces (SQLite)

QUALITY

Quality is not requested: it is executed

The stack runs its own controls on every commit, every push and every health cycle.

30

Quality gates

28 of 30 enforced strictly

272

Test files

Unit, e2e, integration, security and skills

21

CI workflows

CI, security, release, public sync and automated labelling

80

Secret patterns

With optional entropy and redaction by default

INSTALL

From clone to first session

Idempotent steps: if it breaks, you re-run it and nothing duplicates.

01 02 03 04

Prerequisites

Node 22.18 or later, pnpm 11 and Git. Strict verification with install:doctor.

Dependencies

pnpm install with a frozen lockfile. The workspace tree resolves once.

Nexus

The SQLite database is created under .runtime and pending migrations are applied.

Hooks

lefthook is installed with the commit and push hooks.

The router decides, and the code decides it

Temperature and the default model are locked by policy: they do not change on the fly.

Criterion	Current rule
Profiles	cheap, balanced and premium. The active one is balanced.
By domain	premium for finance, legal and governance; balanced for marketing, sales, HR and telecoms; fastCheap for gitflow, ops and session.
Escalation	Steps up by input length, item count, or architectural design and security review questions.
Failover	opencode, then ollama, then lm-studio2, with a 5 minute cache and a fallback model.
Circuit	Five failures open the circuit, two successes move it to half-open.
Temperature	Locked by default. Command override is not permitted.
Audit	Every routing decision is written to a log with 90 days of retention.

RULES

What is not negotiable in this repository

Invisible processes

Every spawn goes through runNpxTsx with windowsHide. Zero console windows. A regression test verifies it.

Documentation with order

Summary, context, scope, out of scope, assumptions, solution, operation, data, security, plan and annexes. Never "TBD".

Destruction guardrail

No bulk deletion or overwrite without explicit confirmation. The untracked snapshot runs before any risky operation.

Commits per slice

Large work: feature doc before the first write, slices of roughly 400 lines and one conventional commit per slice.

TypeScript and bash only

Stack logic goes in TS. PS1 and CMD only in sanctioned exceptions. No new PowerShell scripts.

Reviewed boundary

After an accepted review, the next one starts from the delta rather than the full range.

REFERENCE

Where to find each kind of answer

The knowledge is versioned. You do not need to ask.

I need	Go to
The rule currently in force	AGENTS.md at the repository root
The detail of one module	docs/stack-manual-modules.md
The complete architecture	docs/stack-manual-full.md
Generated stack figures	docs/generated/STACK-FACTS.md
Task examples with agents and skills	docs/use-cases/EXAMPLES.md
Project, proposal and contract templates	docs/templates/gv/
How the design system stands	packages/gv-design-system and docs/brand/
The interactive presentation book	docs/presentations/

GentleVanguard

Your first day

Install, start a session and ask for the stack health status. Then pick a small issue and let the inline flow do the rest: explore, implement and verify in proportion to the risk.

Start with setup

See the command map

Documentation: AGENTS.md and docs/stack-manual-modules.md

Setup command: npm run setup

Health status: npm run watchtower:health

If you have any doubt about a repository rule, the answer is in AGENTS.md or in the stack manual.